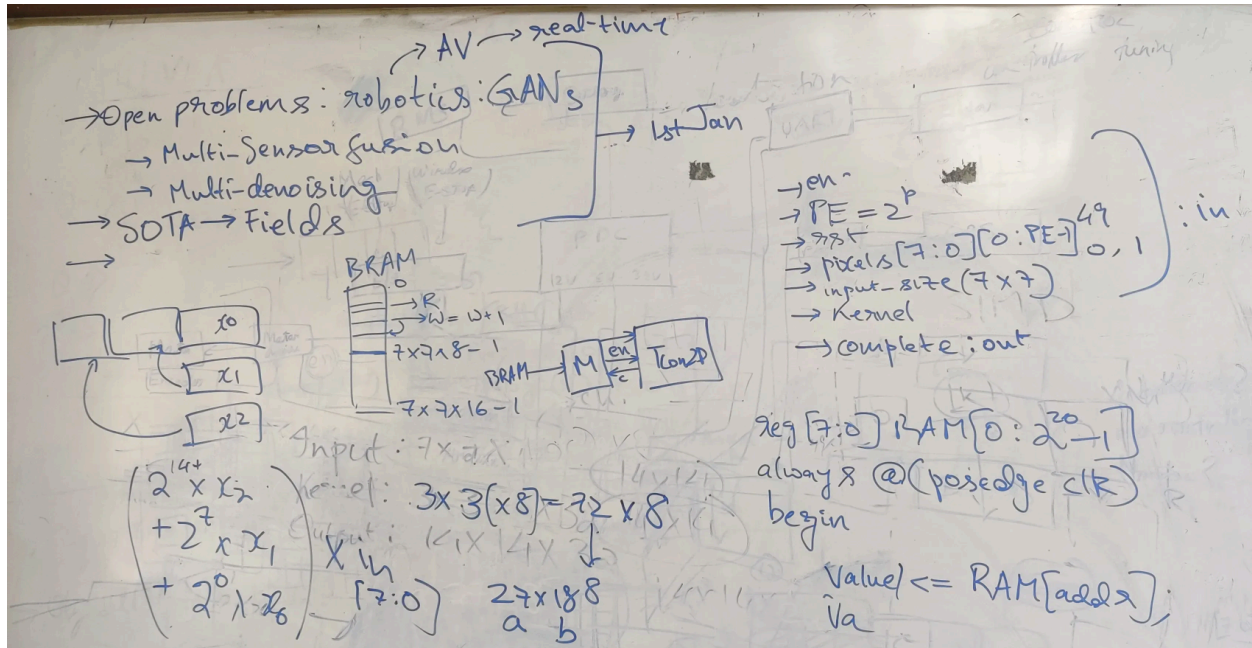


Task 2

Task 2 PS



- We have to implement Deconv2D for multiple images. Lets say image is N*N with some channels K, for eg: 7*7*16, so now we need to implement the whole Deconv2D for 16 images and then accumulate these images into one higher resolution image.
- We are accumulating using 8 cores and in each core you accumulate for all channels by adding index i with i+ size i+2*size and so on.

Important Universal Conventions and Notations while Coding

- No always block inside generate block
- n bit input pixel values, 2n bit for pixel*kernel_weight, 4n bit final output.
- parameters: **K** = Maximum kernel width, **N** = Image width.

Architecture Overview

- What we are doing is that we use our multiply, decode and accumulate modules such that they each take in a pixel and a kernel weight and perform all operations per weight. This entire cycle of operations resides in a generate block which does this for all the weights in parallel, implying that our multiply and decode modules operate byte-wise.

Modules Used:

1. **Multiply Unit:** Here we do byte wise multiplication of the pixel value and kernel weight and gives a 16 bit output.
2. **Decoder:** Here we do the index decoding of assigning the elements of the multiplied matrix and obtain a decoded index.
3. **Deconv2D:** This is the Sub top module.
 - a. This module has 4 states in its FSM that control the flow:
 - i. **IDLE:** Sets the Deconvolution to idle mode, it also listens for the enable pulse.
 - ii. **CLEAR_RAM:** Clears the result_RAM and kernel_RAM before starting any computation.
 - iii. **INITIALIZE:** Loads the kernel weights into the kernel matrix. This state is used only once.
 - iv. **ASSIGN_REG:** Assigns a temporary register to the value of the multiplied output wire.s
 - v. **ADD:** When we enter this state, we have our outputs ready. Here we perform the accumulation part of a Transpose Convolution.
 - vi. **DONE_STATE:** Signals to the sender that one round of image multiplication with the kernel is complete.

Documentation

- The code starts with a generate block which instantiates the multiply and decode modules nine times, in parallel.

- Now comes the always block that is triggered at the positive clock edge.
 - Reset just sets everything to 0.
- The flow of the state machine is such that, it starts with the IDLE state, then depending upon an enable pin, it moves on to the CLEAR_RAM state, after this the kernel gets initialized. After initializing and the ASSIGN_REG state, the registers have the correct outputs from the modules stored in them, so we move to the ADD state which performs the accumulation step done in transpose convolution. The ADD state happens for each pixel from the image and once that's done, the FSM moves on to the DONE state and sends the done signal and moves back to the IDLE state in order to wait for the next enable signal.
- There is a ready pin which indicates when the core is ready to read an input, i.e., any computation that was scheduled to occur is complete and the system is ready to read input data. This was done by setting the ready pin to high or low as and when required.

Issues Faced

- Faced issues with excess LUT utilization with our initial approach, thus increasing the synthesized area. We were using a lot of wires and address decoders which are expensive on area consumption.
- So, we completely changed our approach to a mix of parallel and sequential computing such that the synthesized area was minimized with minimal sacrifice in the latency.
- Initially the code worked only for the case of `kernel_width = K`. But then we fixed that by, filling the kernel_RAM, sequentially, and then indexing it in the correct order based on the given kernel_width, by using the equation, **`actual_index = ((weight_counter/kernel_width)*K) + (weight_counter % kernel_width);`**
- Faced an issue of not being able to know when the computation for one pixel is complete, so we added a "ready" signal.

Code

```

module multiply_unit #(
    parameter pixel_bits = 8
)(
    input wire [pixel_bits-1:0] pixel,
    input wire [pixel_bits-1:0] kernel_weight,
    output wire [pixel_bits*2 - 1:0] multiplied_output
);
    assign multiplied_output = (pixel * kernel_weight);
endmodule

```

```

module decoder#(
    parameter K = 3,
    parameter N = 2
)(
    input wire [$clog2(K*K)-1:0] current_index,
    input wire [$clog2(N*N)-1:0] pixel_number,
    input wire [$clog2(K)-1:0] stride,
    input wire [$clog2(K)-1:0] kernel_width,
    output wire [$clog2(N*N*K*K)-1:0] decoded_index
);

    wire [$clog2(K)-1:0] k_x = (current_index / K);
    wire [$clog2(K)-1:0] k_y = (current_index % K);

    wire [$clog2(N*K)-1:0] shift_x = k_x + stride * (pixel_number / N);
    wire [$clog2(N*K)-1:0] shift_y = k_y + stride * (pixel_number % N);

    assign decoded_index = shift_x * (N*K) + shift_y;

endmodule

```

```

module deconv2D #(
    parameter N = 2,

```

```

parameter K = 3,
parameter pixel_bits = 8

)(
    input wire clk,
    input wire enable,
    input wire rst,
    input wire strobe_signal_pixel,
    input wire strobe_signal_kernel,
    input wire [pixel_bits-1:0] pixel,
    input wire [$clog2(K)-1:0] stride,
    input wire [$clog2(K)-1:0] kernel_width,
    input wire [pixel_bits-1:0] kernel_weight,
    input wire [$clog2(N*N)-1:0] pixel_number,
    input wire [$clog2(N*K*N*K)-1:0] result_address,
    output wire [pixel_bits*4-1:0] final_output,
    output wire ready,
    output wire done
);

integer j, k, l;

reg [2:0] state;

reg [pixel_bits-1:0] kernel_RAM [0: K*K-1];
reg [pixel_bits*4 - 1:0] result_RAM [0:N*K*N*K-1];

reg [$clog2(K*K)-1:0] weight_counter;
reg [$clog2(K*K)-1:0] add_counter;
reg [$clog2(N*N)-1:0] pixel_counter;

reg [pixel_bits*2 - 1:0] multiplied_output_reg [0:K*K -1];
reg ready_temp;
reg done_temp;

wire [pixel_bits*2 - 1:0] multiplied_output [0:K*K -1];

```

```

wire [$clog2(N*K*N*K)-1:0] decoded_index [0:K*K -1];
wire [$clog2(K*K)-1:0] actual_index = ((weight_counter/kernel_width)*K) +
(weight_counter % kernel_width);

```

```

assign ready = ready_temp;
assign done = done_temp;
assign final_output = result_RAM[result_address];

```

```

localparam IDLE = 3'b000;
localparam CLEAR_RAM = 3'b001;
localparam INITIALIZE = 3'b010;
localparam ASSIGN_REG = 3'b011;
localparam ADD = 3'b100;
localparam DONE_STATE = 3'b101;

```

```

genvar i;
generate

```

```

    for (i = 0; i < K*K ; i = i + 1) begin : module_instances

```

```

        wire [$clog2(K*K)-1:0] idx_wire = i;

```

```

        multiply_unit #(
            .pixel_bits(pixel_bits)
        ) multiply(
            .pixel(pixel),
            .kernel_weight(kernel_RAM[i]),
            .multiplied_output(multiplied_output[i])
        );
        decoder #(
            .N(N),
            .K(K)
        ) decode_module(
            .current_index(idx_wire),
            .pixel_number(pixel_number),
            .stride(stride),

```

```
        .decoded_index(decoded_index[i])  
    );  
end  
  
endgenerate  
  
always @(posedge clk) begin  
end
```

Verilog Reference Guide